Clerk Migration Plan — Build 28

Status: **Planning. No code changes yet.**

Strategy: **Lazy / just-in-time migration** (Option A from the rollout discussion).

Constraint: Production sees new signups daily — zero-downtime, zero-orphan migration required.

Goal

Replace the custom email/password + JWT auth stack with **Replit-managed Clerk** so that:

- Apple sign-in and Google sign-in work end-to-end (currently silent no-ops on the signup screen)
- Email verification is enforced for new accounts
- The 71 existing prod users keep working without ever being forced to reset their password
- Their RevenueCat fleet entitlements survive the migration intact
- New signups that arrive *during* the rollout are not orphaned

---

Why "lazy migration" and not bulk import

A bulk import (snapshot the `users` table → push into Clerk → ship) creates an orphan window from the day of import to the day TestFlight launches. With daily signups + Apple review (24–48h), this window is days, and any user who signs up inside it would be locked out of the new build. Lazy migration eliminates the orphan window entirely.

Source-of-truth contract (the most important decision)

After the migration:

- **Our `users` table remains the source of truth** for `id`, `name`, `email`, `country`, `is_admin`, `password_upgrade_required`, foreign-key relationships to `cars` / `maintenance` / `password_reset_tokens` / etc.
- **Clerk is the source of truth for credentials only** — what password they have, which OAuth providers they've linked, whether their email is verified, session tokens.
- A new column `users.clerk_user_id text unique` links the two. `NULL` means "not yet migrated" (still on legacy bcrypt auth).
- `users.id` **(the numeric serial) never changes** — this is what RevenueCat already uses as `appUserID`, what every foreign key already references, and what every existing JWT encodes.

This contract is what makes the migration safe: nothing downstream of auth (subscriptions, cars, maintenance, admin checks) needs to know Clerk exists.

Schema changes

Single column add:

```
// lib/db/src/schema/cars.ts — usersTable
clerkUserId: text("clerk_user_id"), // unique, nullable
```

Plus a unique index `users_clerk_user_id_unique` on it. Default `NULL`. No backfill needed — gets populated lazily as users migrate.

That's it. No other schema changes required.

Rollout phases

Phase 0 — Prep (no user impact)

1. Set up Replit-managed Clerk via the Auth pane in workspace
2. Configure Apple + Google OAuth providers in the Auth pane (Replit handles the Apple Service ID / Google client setup; we just enable them)
3. Add the `clerk_user_id` column to dev DB via `pnpm db push`
4. Install `@clerk/clerk-expo` in `artifacts/car-logbook` and `@clerk/express` in `artifacts/api-server`

Phase 1 — Server-side dual auth (no app changes)

The API server learns to accept *either* a legacy JWT (existing 71 users) *or* a Clerk session token, on every protected route.

`requireAuth` middleware becomes:

```
1. Read Authorization header
2. If "Bearer eyJ..." with our JWT format → validate as legacy JWT → req.user = legacy user
3. Else if Clerk session token → validate via Clerk → look up local user by clerkUserId → req.user
4. Else 401
```

Both paths populate `req.user` with the same shape the rest of the app already expects (numeric `id`, `email`, `isAdmin`). Downstream code is unchanged.

This phase is invisible to users — no one is logging in via Clerk yet.

Phase 2 — New endpoints for Clerk-based flows

Add three new routes (legacy `/auth/*` routes stay live):

- `POST /auth/clerk/sync-user` — called by the new mobile app after a successful Clerk sign-in. Takes a Clerk session token, finds-or-creates the matching row in our `users` table, returns the local user object (`id`, `name`, `email`, `isAdmin`, `passwordUpgradeRequired`).

- **Lookup order**: by `clerk_user_id` first, then by `email` (the migration-on-first-login case).

- When matching by email, set `clerk_user_id` on that row → user is now permanently linked.

- When neither matches (brand-new signup via Clerk), insert a new row.

- `POST /auth/clerk/migrate` — called by the new app on legacy users' first Clerk login attempt. Takes their email + the password they typed into the new login form. Server validates against the legacy bcrypt hash; if valid, it programmatically creates them in Clerk via the Clerk Backend API (preserving the

bcrypt hash so they don't notice anything), sets `clerk_user_id` on their row, and returns success. The app then has them sign in to Clerk normally.

- This is the magic step. From the user's POV they just typed their old password into the new login screen and it worked. Behind the scenes we silently moved them into Clerk.

- (Optional) `POST /auth/clerk/link` — for users who want to add Apple/Google to their existing email account from settings.

Phase 3 — New TestFlight build

Mobile app changes:

- Login screen: tries Clerk sign-in first. On failure with "user not found in Clerk," call `/auth/clerk/migrate`, then retry Clerk sign-in. Net UX: same login form, same password, no visible difference for the 71 existing users.
- Signup screen: the existing strict-rules form now creates the account directly in Clerk (with email verification step), then calls `/auth/clerk/sync-user` to mirror it locally.
- Apple / Google buttons: now functional via Clerk's OAuth flows.
- AuthContext: instead of storing our JWT, stores the Clerk session and asks Clerk for the current user. Then calls `/auth/clerk/sync-user` to get the local user object (which is what every other screen reads).
- API requests: Authorization header now uses Clerk session token instead of our JWT. The dual-auth middleware (Phase 1) accepts either, so this is a clean swap.

Phase 4 — Cleanup (weeks later, separate build)

Once all 71 users have logged in at least once and `clerk_user_id` is populated for them (we'll have a query to check):

- Drop the legacy JWT path from `requireAuth`
- Remove `/auth/login`, `/auth/register`, `/auth/me`, `/auth/change-password`, `/auth/forgot-password`, `/auth/reset-password` (mobile no longer calls them)
- Remove `passwordHash` and `passwordUpgradeRequired` columns from `users` table
- Remove `password_reset_tokens` table
- Remove the `PasswordUpgradeModal` (no longer relevant — Clerk handles password policies)

For users who never log in (estimated <5 of the 71), leave them with `clerk_user_id = NULL` indefinitely. They effectively become dormant accounts. We can email them later if needed.

RevenueCat preservation strategy

This is the highest-risk part of the migration because subscription revenue is at stake.

Current behavior (`lib/revenuecat.tsx:148`):

```
Purchases.logIn(String(user.id))
```

RevenueCat's `appUserID` for the 71 users is their numeric `users.id` (e.g. `"11"`, `"13"`, `"14"`).

After migration: keep using `String(user.id)` exactly as today. Because we never change the numeric `id` (only add `clerk_user_id` alongside it), every existing user's `appUserID` in RevenueCat stays identical. Their entitlements transfer with zero RevenueCat-side changes.

For new Clerk-native signups: same code path — `/auth/clerk/sync-user` returns a user object with our local numeric `id`, and the RevenueCat hook calls `Purchases.logIn(String(user.id))` with that.

Verification gate before TestFlight ships: write a quick script that picks 3 of the 7 dev test accounts, runs them through the full migration flow, and confirms their RevenueCat customer info (entitlements + subscription status) is unchanged.

Rollback plan

Each phase is independently revertible:

- **Phase 1 (dual-auth middleware)**: revert the middleware change → server only accepts legacy JWTs again. New build app keeps working because it falls back to `/auth/clerk/migrate` which keeps validating against bcrypt hashes.
- **Phase 2 (new endpoints)**: leaving these live is harmless — they're only called by the new app build.
- **Phase 3 (new TestFlight build)**: if catastrophic, expire the TestFlight build / pull from review. Users on the old build continue working through legacy auth (which is still live).
- **Phase 4 (cleanup)**: don't do this until Phase 3 has been stable in TestFlight + App Store for 2+ weeks. Easily skipped/delayed.

Open questions to resolve before Phase 0

1. **Clerk Apple sign-in setup**: Replit-managed Clerk handles the OAuth provider config, but Apple sign-in requires the bundle ID `com.ctl.carlogbook` to be registered with the right Service ID in Apple Developer console. We need to verify this works before committing to the new build. (~15 min check via Clerk Auth pane.)
2. **Email verification for the 71 existing users**: Clerk by default requires email verification for new sign-ups. When we silently migrate an existing user via `/auth/clerk/migrate`, do we mark their email as already-verified (since they've been receiving forgot-password emails for months and it works)? Recommendation: yes, mark as verified — they've proven email ownership through prior use.
3. **Clerk MAU pricing** at our scale (71 today, growing daily). Need to check the current Replit-managed Clerk pricing tier limits before going further.
4. **Test account cleanup**: the 7 dev test accounts (`testuser@example.com`, `mkashour@gmail.com (Test)`, 2× Test (icloud), `mkashour@icloud.com`, `mkashour.apple@gmail.com`) should be cleaned from prod before migration so we're not migrating junk into Clerk.

Estimated effort

- Phase 0: 1 session
- Phase 1: 1 session
- Phase 2: 1–2 sessions
- Phase 3: 2–3 sessions (mobile UX + thorough testing)
- Phase 4: 0.5 session (weeks later)

Total: ~5–7 working sessions before the new build is ready for TestFlight.

What this plan does NOT do

- Does not touch the password upgrade modal we just shipped — it stays live and stops being shown automatically once Clerk takes over (Phase 4 removes it).
- Does not change RevenueCat product IDs, offerings, or entitlements.
- Does not migrate Replit Auth users (we're not on Replit Auth — we're on custom JWT).
- Does not implement organization/team features for "CTL Enterprise" — that's a separate piece of Build 28 that can build on top of this.